Week 11: NoSQL

Keeyong Han

Table Of Contents

1. Recap of the 10th Week
2. What is NoSQL?
3. Embedding Introduction
4. VectorDB Overview
5. Pinecone Overview
6. Break
7. Article Recommendation with Pinecone
8. Demo & Homework #8
9. Lab #2
10. Quiz #3

Recap of the 10th Week

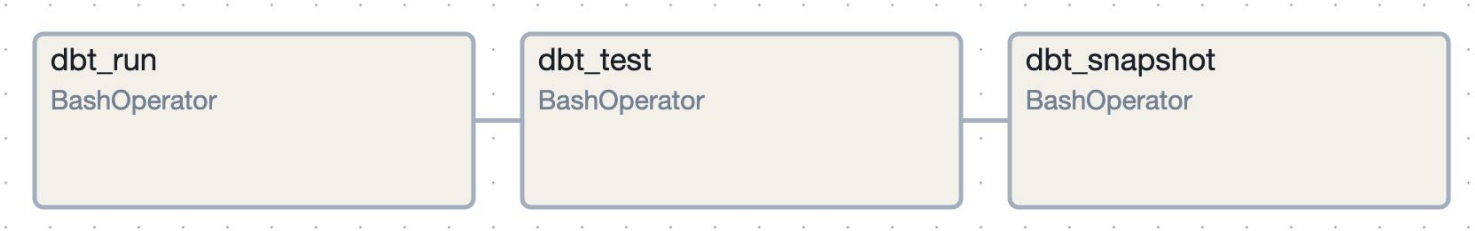
dbt Project: Building analytics.session_summary

- dbt: In-warehouse Transformation
  - dbt Core vs. dbt Cloud
- dbt Project Code
  - sources.yaml: defines source tables (ETL) and add documentation
    - “{{ source('raw', 'session_timestamp') }}” is the notation to use
  - schema.yaml: add documentation and tests to tables created by dbt
    - “{{ ref("session_timestamp") }}” is the notation to use

Schedule the dbt tasks in Airflow

1. Set up a dbt project folder as the same level of dags folder
2. Mirror the folder with Airflow docker container (docker-container.yaml)
3. Add dbt-snowflake python package to your docker-container.yaml
4. Build a DAG to schedule different dbt jobs

- Source Code
- This DAG is needed in **the lab 2** (and possibly in the group project)



What is NoSQL?

Intro to NoSQL

- Non-relational databases designed to handle large volumes of diverse data
  - Denormalized schema and NO JOIN ~~(Star Schema)~~
- Provides flexibility in data schema design, unlike traditional SQL databases
- Examples:
  - MongoDB, Cassandra, Redis, CouchDB, ElasticSearch, Vector Database
- Key Characteristics:
  - Schema-less or loose schema: Data structures can vary, making it easier to adapt to changes.
  - Scalability: Built for horizontal scaling (Scale Out), allowing easy addition of nodes for handling large datasets or large traffic
  - Distributed: Data can be spread across multiple servers, improving performance & availability.
  - High Performance: Optimized for fast reads and writes, ideal for real-time applications.

NoSQL vs. SQL Databases

- Data Model
  - SQL: Tables with rows and columns (fixed schema).
  - NoSQL: Documents, key-value pairs, graphs, or wide-column stores (flexible schema).
- Scalability
  - SQL: Vertical scaling (increasing server power - Scale Up).
  - NoSQL: Horizontal scaling (adding more servers - Scale Out).
- Consistency
  - SQL: Strong ACID (Atomicity, Consistency, Isolation, Durability) compliance.
  - NoSQL: BASE (Basically Available, Soft state, Eventual consistency).
- Use Cases
  - SQL: Financial transactions, enterprise applications.
  - NoSQL: Real-time analytics, IoT, social media, content management.

Types of NoSQL

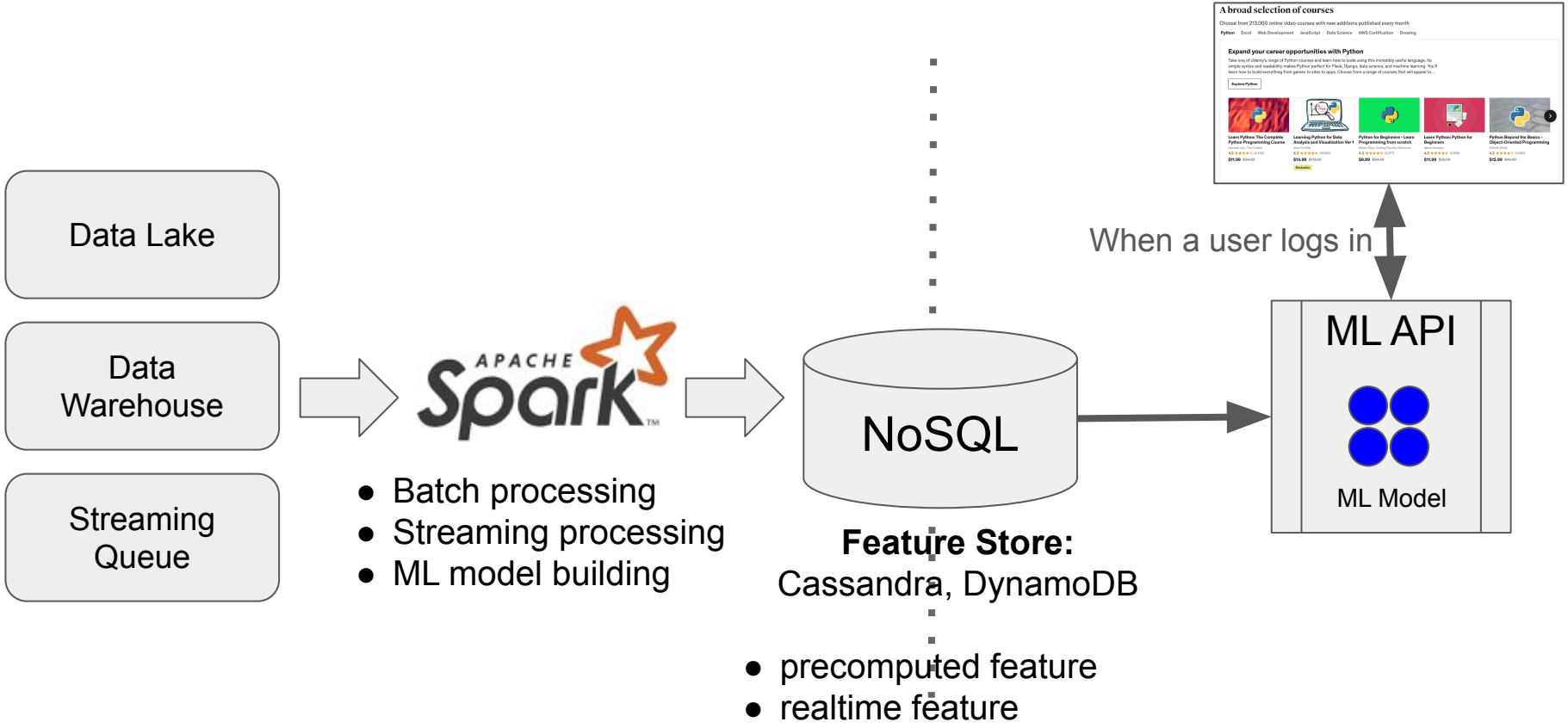
- Simple Key/Value Storage
  - Very fast but limited in lookup
  - Redis and Memcached (both are in-memory) -> AWS ElastiCache
- Document Store
  - A fancy key-value storage with many lookup options. Values are semi-structured (JSON,Blob)
  - A good fit if your data has hierarchical structure
  - MongoDB is the most popular (AWS DocumentDB)
- Wide Column Storage
  - Somewhere between a document store and a relational database
  - Uses tables/rows/fields but each field can be big and structured
  - AWS DynamoDB, Cassandra, HBase, BigTable, ...
- Search Engine (similar to Document Store)
  - ElasticSearch, VESPA, **PineCone**, …
  - Index Search, **Vector Search**, Hybrid Search

Use Cases of NoSQL

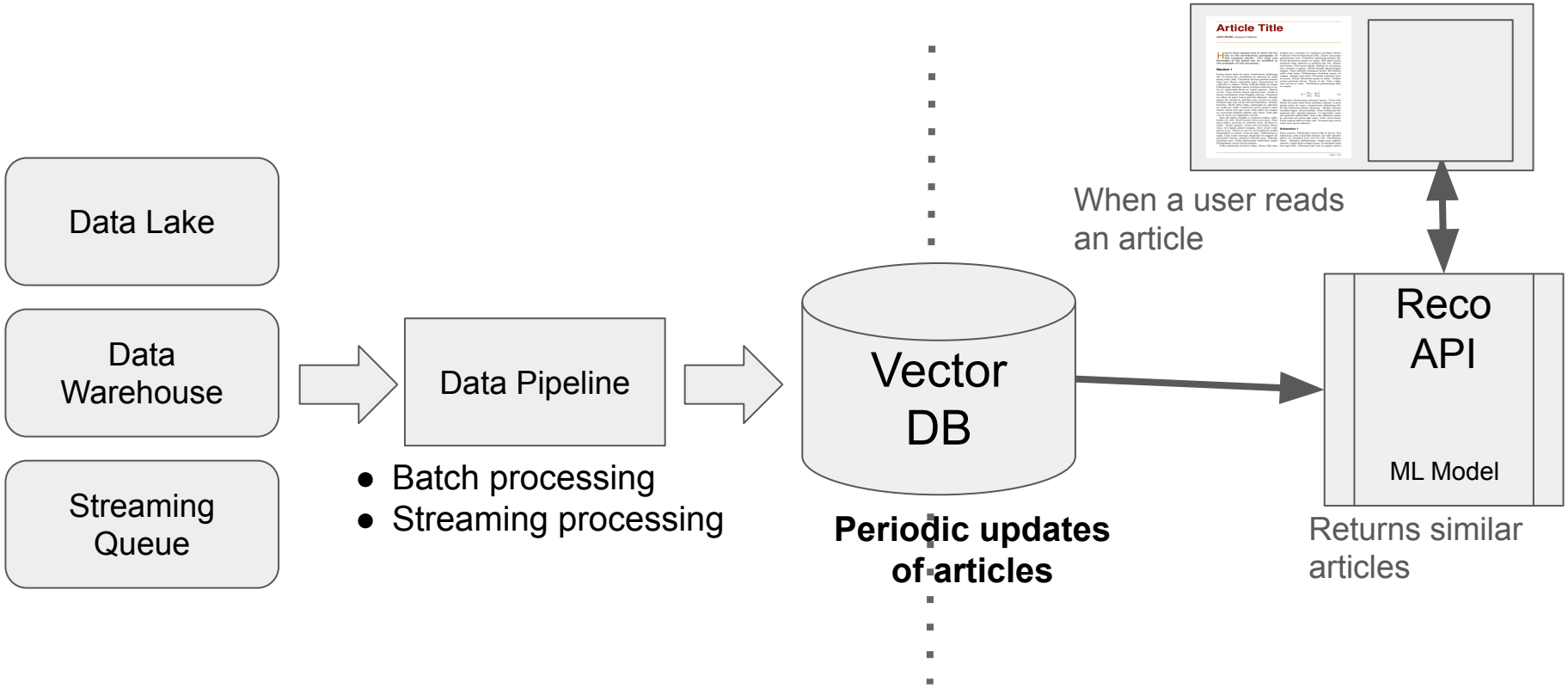
- Use NoSQL when:
  - You're pulling many single records quickly from production setting
    - Response time needs to be short
  - The records' schema needs to be very flexible (might need to change over time)
- Real-Time Analytics
- Recommendation Engines
- Precomputed Feature Storage for ML System

- In many cases, the input to the NoSQL systems comes from Data Warehouse, Data Lake or Streaming System ("Kafka")

A Use Case of NoSQL: Feature Store for ML System



A Use Case of NoSQL: Similarity based Recommendations



Embedding Introduction

What is Embedding?

- Definition:
  - An embedding is a representation of high-dimensional and sparse data in a lower-dimensional and dense space, where each data point is represented as a vector (a list of numerical values).
  - This transformation is often learned through deep learning that are trained to capture the underlying semantics of the data.
- Purpose:
  - Dimensionality Reduction
  - Similarity Measurement

Common Types of Embeddings

- Word Embeddings
  - Transform words into vectors, capturing semantic meanings
  - Examples include Word2Vec, GloVe, and BERT.
- **Sentence Embeddings**
  - Represent sentences or paragraphs as vectors, capturing the overall meaning of the text (e.g., Sentence-BERT).
- Image Embeddings
  - Use convolutional neural networks (CNNs) to transform images into vector representations, capturing features like edges, textures, and object shapes.
- User and Item Embeddings
  - Represent users and items (e.g., products or movies) in recommendation systems based on their interactions.

Applications in ML

- Natural Language Processing (NLP)
  - Word and sentence embeddings are used for text classification, sentiment analysis, and translation.
- Recommendation Systems
  - User and item embeddings help match users with items based on their preferences and past behavior.
- Clustering and Visualization
  - Embeddings make it easier to cluster similar data points and visualize high-dimensional data using methods like t-SNE or UMAP.

Word Embedding Demo

- Gensim Word Embedding Test using Word2Vec-google-news

Vector DB Overview

Issues with Traditional Keyword matching Search

- If the exact words are not matched, good results may not be produced
  - “shiny” vs. “bright”
- To perform synonym searches, a synonym dictionary must be explicitly built.
- It is not easy to recognize entities.
  - For example, understanding that "New York" is essentially a single term is challenging.
- It only supports text searches.

Vector Search to the Rescue

- Use of embeddings reduces the dimensionality of the data.
- It enables semantic search (not exact word matching)
  - Exact Matching -> Similarity Measurement
- It can support various types simultaneously
  - Text, image, audio and so on
  - This is known as multimodal search
    - CLIP supports both text and image embedding (Universal Embedding)

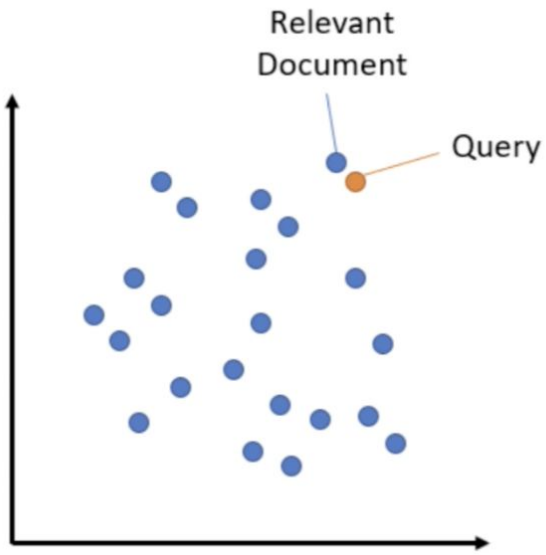
Use Cases of Vector Search (or Vector Database)

- Semantic Search & Similarity Search
- Long-term Memory (RAG) for Assisting LLMs
  - Used for Providing Supporting Data (“Domain Knowledge”) in Prompts
- Recommendation Systems
- Machine Learning
  - Clustering and Classification Systems
- Anomaly Detection

Advantages of Vector Databases: Semantic Search

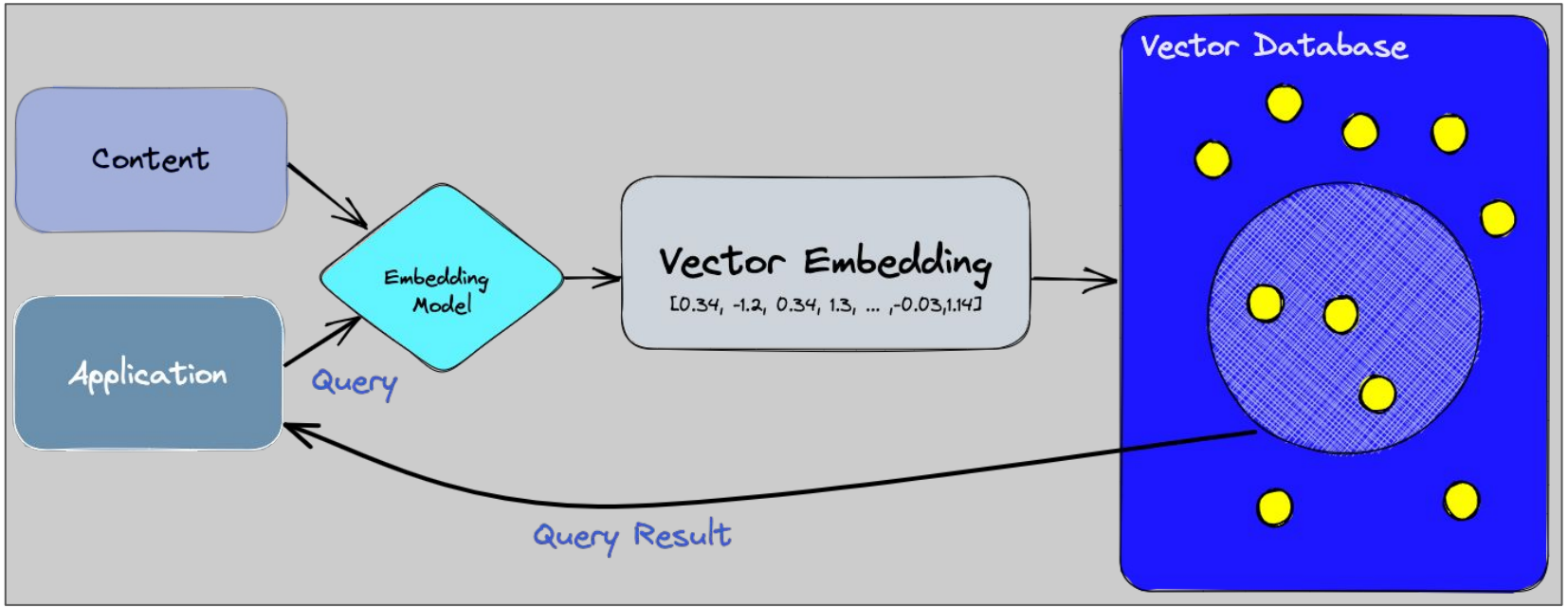
- Search results are improved.
  - This is because it can more easily identify synonyms and related terms
  - bright vs. shiny
- What embedding to use is important

1. Convert all documents into vectors and save them in the Vector database.
2. Convert the search query into a vector.
3. Search for other neighbors in the vector database that are approximate to the search vector (KNN).
4. The results of step 3 are the search results.



Advantages of Vector Databases: Multi-Modal Search

- Indexing and searching of various data types are possible

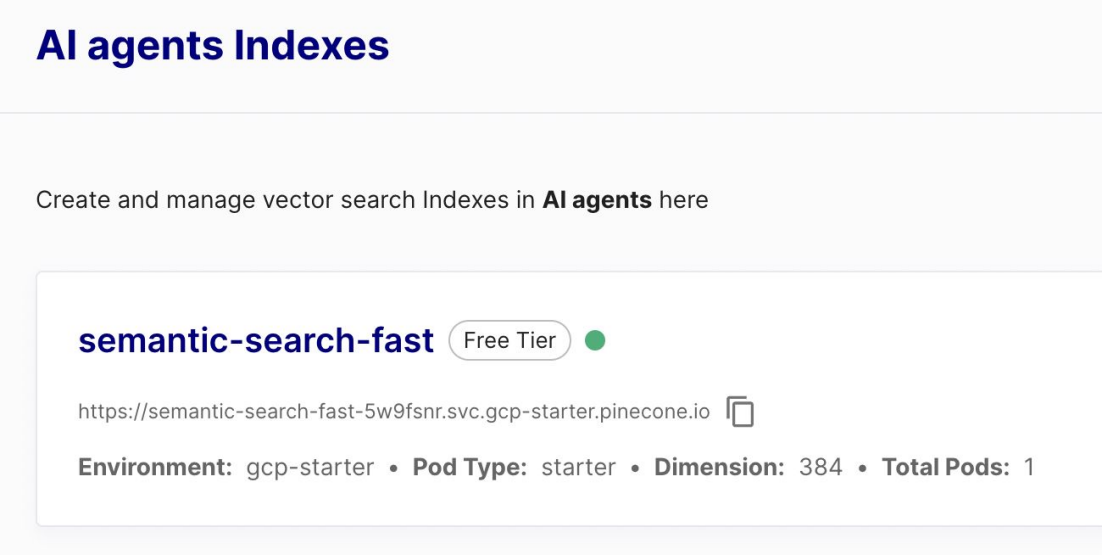


Vector Database Options

Pinecone Overview

What is Pinecone?

- Pinecone is a cloud service that makes it easy to implement similar content search using vector embeddings
  - https://www.pinecone.io/
- We plan to use the free tier.



Pinecone’s Index Model

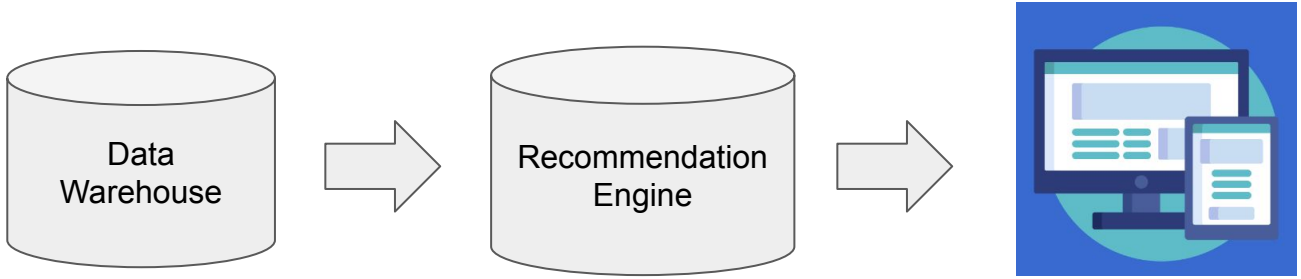
- PodSpec vs. ServerlessSpec
  - We will be using ServerlessSpec
- ServerlessSpec
  - You don’t manage compute resources like CPU, memory, or pod count.
  - Auto-scaled: Resources automatically scale up or down based on demand.
  - Pricing is based on the number of stored vectors and query volume.
- PodSpec
  - You manually configure the pod_type (like p1.x1, s1.x1, etc.) and the number of replicas.
  - Predictable performance: Control over latency and throughput.
  - Pricing is based on the number and type of pods, billed at a fixed rate.

Break

Article Recommendation with Pinecone

What We Are Going to Build: Article Recommendation

- Dataset: Medium Article
- Content based Recommendation: Purely based item's content similarity
  - Other types of recommendations: Item based, User based, Collaborative Filtering
- Tech stack to use in production
  - Snowflake or Kafka
  - **Pinecone** or VESPA or ElasticSearch
  - Airflow
    - A DAG to read the latest Medium article data into Snowflake (medium table)
    - A DAG to process the medium table records and push to Pinecone

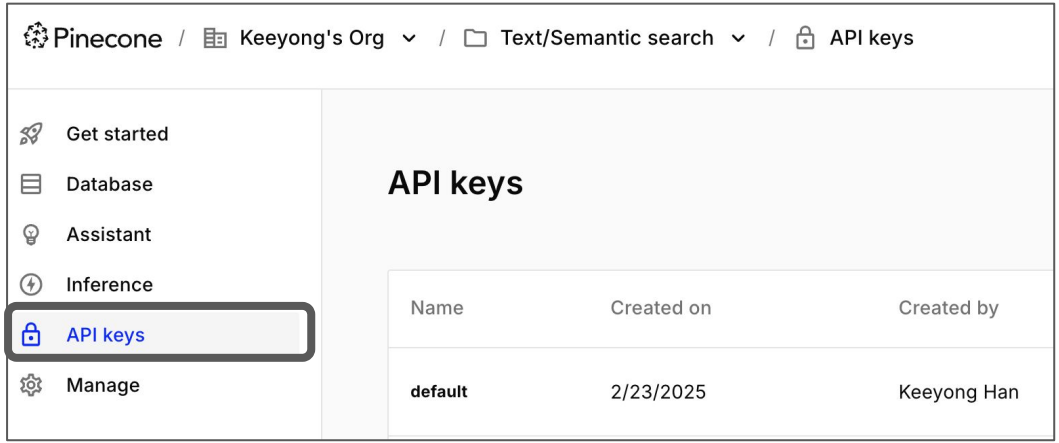


What we are going to practice (Medium article -> Pinecone)

1. Set up Pinecone Account and API key
2. Download Medium article data (csv file)
3. Preprocess Medium data
4. Set up Pinecone Index
5. Generate sentence embeddings from step 3
6. Create a Dataframe for Index Ingestion
7. Ingest the Dataframe into Pinecone
8. Run search against Pinecone index

1. Set up Pinecone Account and API key

- Visit https://www.pinecone.io/
- Two types of indexes exist: Pod-based & Serverless
  - Pod-based option: Allocate capacity in dedicated fashion and use it
  - Serverless option: Use resources in elastic fashion
- As of Apr 2025, limitations of free tier are:
  - Serverless is the only option
  - 2GB storage
  - 2M writes per month
  - 1M reads per month
- Copy API token for the demo



2. Install packages and Download Medium article data

- Install two Python packages:
  - sentence-transformers
  - pinecone
- A static CSV file will be used
  - https://www.kaggle.com/datasets/arnabchaki/medium-articles-dataset
  - We will use a subset of this dataset:
    - https://s3-geospatial.s3.us-west-2.amazonaws.com/medium_data.csv
- You can also use other datasets such as movie dataset
  - https://grepp-reco-test.s3.ap-northeast-2.amazonaws.com/tmdb_5000_movies.csv

3. Preprocess Medium data

- Lambda Function in Python
- Create a new column named “metadata”
- Populate it with title + subtitle using Pandas apply function
  - df['metadata'] = df.apply(lambda row: {'title': row['title'] + " " + row['subtitle']}, axis=1)
    - **axis=1** is needed when the function needs to access multiple columns

4. Set up Pinecone Index

- Access ServerlessSpec
- Create an index named 'semantic-search-fast'
    - This will have 3 fields:
        - id: Unique ID of each record. Should be a string type
        - values: Sentence embedding vector
        - metadata

```
pc.create_index(
    index_name,
    dimension=384,  # dimensionality of minilm
    metric='dotproduct',  # dotproduct, cosine, euclidean
    spec=spec
)
```

5. Generate sentence embeddings

- Download all-MiniLM-L6-v2 embedding model from HuggingFace
  - https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2
- Apply sentence embedding against “title” subfield of “metadata” field from Step 3 and create values field.
  - If the text is longer, we should split the text into a number of overlapping chunks

```
df['values'] = df['metadata'].apply(lambda x: model.encode(x['title']).tolist())
```

df['values'] becomes 384 dimensional vector (a list of 384 elements)

6. Create a Dataframe for Index Ingestion

- Create a Dataframe named df_upsert with 3 columns

```
# Create a new dataframe with 3 columns ('values', 'metadata', 'id')
df_upsert = df[['id', 'values', 'metadata']]
# Convert the type of id from numeric to string
df_upsert['id'] = df_upsert['id'].astype(str)
```

7. Ingest the Dataframe into Pinecone

- Perform **Upsert** operation from dataframe to Pinecone Index directly

```
index = pc.Index(index_name)
index.upsert_from_dataframe(df_upsert)
```

8. Run search against Pinecone index

- Convert user’s query into the same sentence embeddings
- Look for top 10 similar articles along with metadata

```
xc = index.query(
    vector=(model.encode("what is ethics in AI")).tolist(), # python list
    top_k=10,
    include_metadata=True,
    include_values=True
)
```

Putting everything together

- Google Colab Example
- Airflow DAG Example

Demo & Homework #8

Demo

- Let’s learn what MongoDB is when it can be useful

MongoDB, Atlas & Compass — How They Work Together

🧠 **MongoDB** *where your data actually lives.*

- The **core NoSQL database**
- Stores data in **collections** (like tables) → each record is a **JSON document**
- Flexible schema — easy to scale and modify

☁️ **MongoDB Atlas** *MongoDB running on the cloud.*

- **Cloud service** for hosting MongoDB
- No installation or server setup needed
- Provides security, backups, and scaling automatically

💻 **MongoDB Compass** *a visual dashboard for your database.*

- **GUI tool** (desktop app) to connect to MongoDB or Atlas
- Lets you **view, query, insert, update, and delete** data visually
- Perfect for beginners & demos

MongoDB Structure: From Top to Bottom

Connection → Database → Collection → Documents → Fields

1. **Connection - Link** between your Compass app and your MongoDB server (either local or Atlas).
2. **Database -** A **database** is a container (like a folder) for related collections.
3. **Collection -** A **collection** is like a **table** in SQL — it stores a group of related documents.
4. **Document -** A **document** is one record — stored in **JSON-like** (BSON) format.

MongoDB vs SQL — The Main Differences

Feature	SQL Database	MongoDB
Data Format	Tables (rows & columns)	JSON-like Documents
Schema	Fixed — must define structure first	Flexible — each document can differ
Query Language	SQL (`SELECT * FROM students`)	Mongo Query Language (`db.students.find()`)
Scalability	Vertical (bigger server)	Horizontal (sharding across many servers)
Relationships	Foreign keys, joins	Embedded documents, references
Use Cases	Banking, ERP, structured data	Apps, analytics, IoT, flexible data

Industry Use Cases of NoSQL

1. **E-commerce: Shopping Cart & Session Storage :** Handles mns of concurrent users, rapid updates, and dynamic schemas.

   - **Tech Example:** Amazon DynamoDB for shopping carts, sessions, and product catalogs.
   - **Key Benefit:** Low-latency reads/writes, horizontal scalability.

2. **Finance: Fraud Detection in Real Time :** Stores large-scale streaming transaction data with flexible schema.

   - **Tech Example:** Cassandra + Kafka for analyzing payment anomalies.
   - **Key Benefit:** Fault tolerance, high write throughput.

3. **Gaming: Player Profiles & Leaderboards :** Fast access to player stats, achievements, and rankings.

   - **Tech Example:** Redis (leaderboards), MongoDB (profiles).
   - **Key Benefit:** Real-time updates with sub-millisecond latency.

4. **IoT & Telemetry: Sensor Data Storage :** Handles billions of time-series sensor readings.

   - **Tech Example:** InfluxDB, Cassandra for storing telemetry.
   - **Key Benefit:** Scalable writes, TTL for auto data expiration.

5. **Social Media: Feeds & Content Discovery:** Flexible schema and fast querying of posts, comments, and user data.

   - **Tech Example:** ElasticSearch (search), Cassandra (feeds).
   - **Key Benefit:** Personalized content retrieval and scalability.

Emerging Trends in NoSQL Databases

1. Multi-Model Databases : Combine multiple data models (document, key-value, graph) under one engine.

- **Examples:** ArangoDB, Cosmos DB, OrientDB.
- **Benefit:** Flexibility — one database fits multiple workloads.

2. Serverless NoSQL : Automatically scales compute/storage without manual provisioning.

- **Example:** DynamoDB On-Demand, FaunaDB, Pinecone Serverless.
- **Benefit:** Pay only for usage; auto handles traffic spikes.

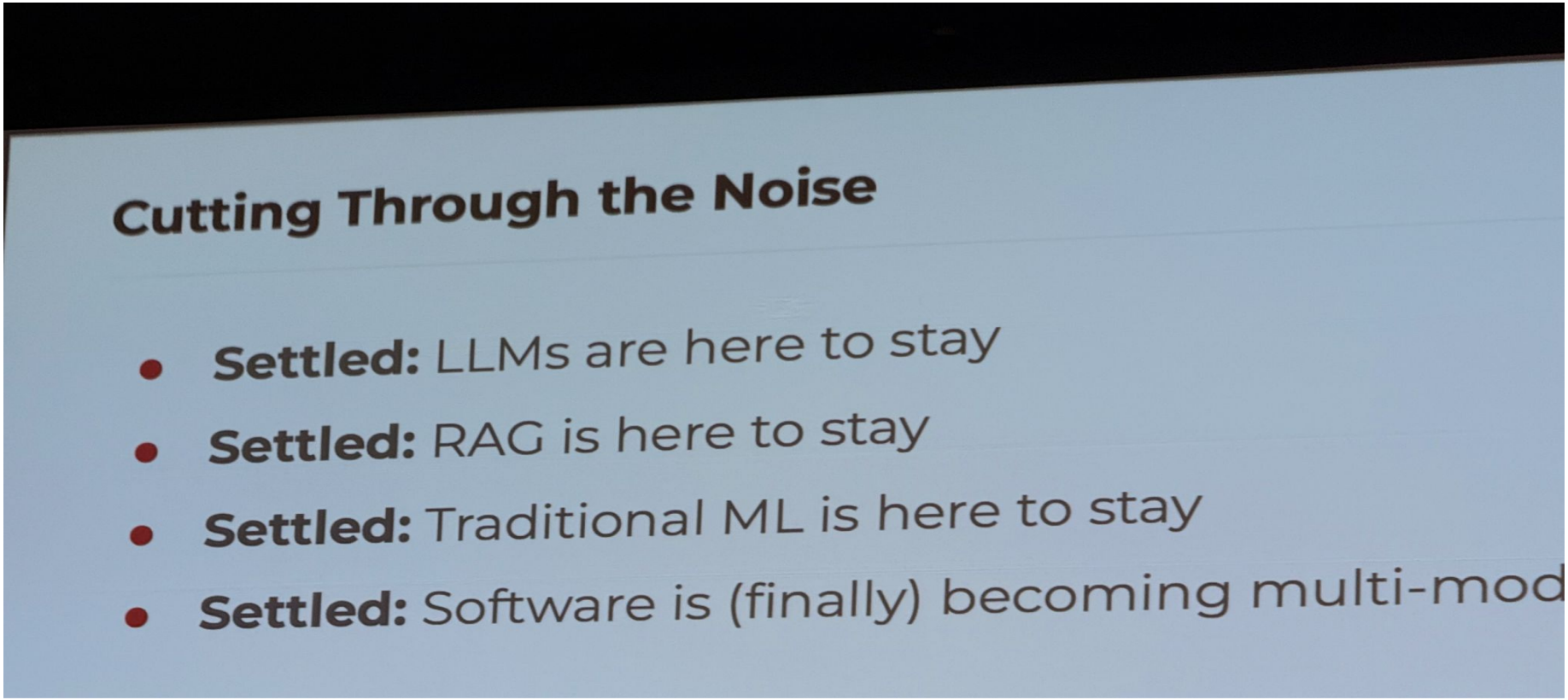
3. Vector + NoSQL Fusion (AI Databases) : Integration of **vector search** with traditional NoSQL for semantic retrieval.

- **Example:** MongoDB Atlas Vector Search, Redis Vector Similarity.
- **Use Case:** Hybrid search in AI chatbots and recommendation systems.

4. Edge NoSQL Databases : Data stored and queried closer to users/devices.

- **Example:** Couchbase Mobile, SQLite + replication layers.
- **Use Case:** Offline-first mobile and IoT apps.

One more thing!



Lab #2

Due date is 3 weeks from now: 11/25

“dbt” should be scheduled from Airflow

Make sure Idempotency (“SQL” transaction)

Homework #8: Due on 11/18

Follow today's class instruction (running Pinecone job as a Airflow job)

1. Modify your docker-compose.yaml file to include sentence-transformers and pinecone packages (1pt)
   - Restart your docker containers ("docker compose down" and "docker compose up")
   - docker-compose.yaml should be a part of your github submission
2. Configure Pinecone (account), get the API token and create Airflow Variable (1pt)
3. Download, Process and Generate an input file to Pinecone (2pt)
4. Create Pinecone index (1tp)
5. Convert the input file into embeddings and ingest them into Pinecone (2pt)
6. Run search against Pinecone (1pt)

- For each step, capture screenshots accordingly
  a. For Airflow tasks, capture the logs screen of each task
- Create a github repo for this and add the link to the report

Quiz #3

11/18

No Class Next Week

Veterans Day

Don’t show up! :)